

Low Level Design (LLD) Document

Web Vulnerability Scanner Application

Written By	Cybersentinel Student
Document Version	1.0
Last Revised Date	26/06/2026

Contents

1. Introduction

1.1 What is a Low-Level design document?

1.2 Scope

2. Component Architecture

3. Module and Code Specifications

3.1 Target Configuration

3.2 Crawler Module (crawler.py)

3.3 HTML Form Parser Module (form_parser.py)

3.4 XSS Detection Module (xss_scanner.py)

3.5 SQL Injection Detection Module (sqli_scanner.py)

3.6 Report Generator Module (report_generator.py)

3.7 Mock Vulnerable Application (vulnerable_app/vulnerable.py)

3.8 Main Scanner Orchestrator (scanner.py)

4. Unit Test Cases

1. Introduction

1.1 What is a Low-Level design document?

The goal of this Low-Level Design (LLD) document is to provide the internal logical design of the actual program code for the Web Vulnerability Scanner. The LLD describes the detailed code-level structure of the system, including module roles, function signatures, input/output parameters, internal algorithms, and dependencies. It translates the high-level architecture into a concrete programming specification, allowing developers to understand, maintain, and extend the codebase.

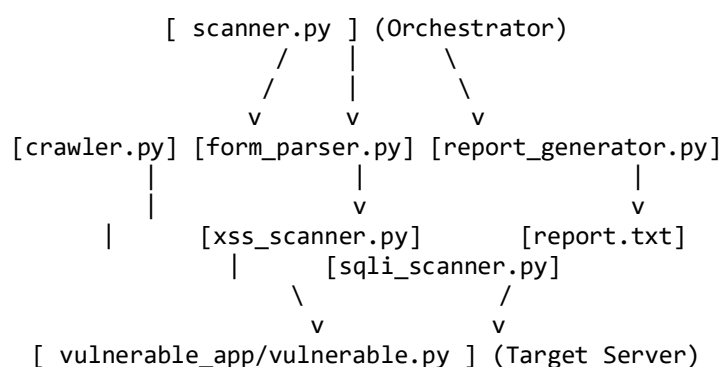
1.2 Scope

This LLD defines the component-level implementation of the vulnerability scanner. It details the following code files:

- `crawler.py`: Hyperlink extraction and web crawling.
- `form_parser.py`: HTML element extraction and form structure mapping.
- `xss_scanner.py`: Cross-Site Scripting signature matching and request handling.
- `sqli_scanner.py`: SQL Injection database error tracking.
- `report_generator.py`: File I/O report writing.
- `vulnerable_app/vulnerable.py`: Mock Flask routes, vulnerable endpoints, and HTML templates.
- `scanner.py`: Command execution flow and orchestration.

2. Component Architecture

The scanner operates as a pipeline. The following diagram shows the structural dependency and code-level flow between the Python scripts:



3. Module and Code Specifications

3.1 Target Configuration

The scanner targets a local web application running over the HTTP protocol.

- **Base URL:** `http://127.0.0.1:5000/`
- **Search URL:** `http://127.0.0.1:5000/search`
- **Login URL:** `http://127.0.0.1:5000/login`

3.2 Crawler Module (`crawler.py`)

Responsible for requesting a target webpage, parsing its DOM, and extracting raw links.

Functions

`get_links(url)`

- **Description:** Downloads the HTML body of a given URL and parses all anchor (`<a>`) tags to find target destinations.
- **Parameters:**
 - `url` (str): The absolute URL of the webpage to crawl.
- **Returns:**
 - `list[str]`: A list of hyperlink target strings found in the page.
- **Dependencies:** `requests`, `bs4.BeautifulSoup`
- **Internal Logic:**
 1. Perform `requests.get(url)` to retrieve the page content.
 2. Create a `BeautifulSoup` object using the "html.parser".
 3. Loop through all tags returned by `soup.find_all("a")`.
 4. Extract the "href" attribute; if valid, append it to the return list.

3.3 HTML Form Parser Module (`form_parser.py`)

Identifies forms embedded in pages to map input parameters for active payload injection.

Functions

`get_forms(url)`

- **Description:** Downloads the HTML document from a URL and extracts form tags.
- **Parameters:**
 - `url` (str): The URL to retrieve.
- **Returns:**
 - `list[bs4.element.Tag]`: A list of BeautifulSoup tag objects representing `<form>` elements.

- **Dependencies:** `requests`, `bs4.BeautifulSoup`
 - **Internal Logic:**
5. Fetch the webpage via `requests.get(url)`.
 6. Instantiate `BeautifulSoup` parsing with `"html.parser"`.
 7. Return the array of tags matched by `soup.find_all("form")`.

3.4 XSS Detection Module (`xss_scanner.py`)

Evaluates input parameters for Reflected XSS vulnerabilities.

Global Variables

- `xss_payload` (str): The test payload string, defined as

```
"<script>alert('XSS')</script>"
```

Functions

`scan_xss(url)`

- **Description:** Sends the XSS payload as a query parameter to the target URL and checks if it is reflected unmodified in the response.
 - **Parameters:**
 - `url` (str): The target endpoint URL (e.g., `http://127.0.0.1:5000/search`).
 - **Returns:**
 - `bool`: True if the payload is reflected, False otherwise.
 - **Dependencies:** `requests`
 - **Internal Logic:**
8. Construct a dictionary `params = {"q": xss_payload}`.
 9. Issue a GET request using `requests.get(url, params=params)`.
 10. Perform a substring search: `if xss_payload in response.text`.
 11. Return True on match, indicating the input is reflected directly.

3.5 SQL Injection Detection Module (`sqli_scanner.py`)

Evaluates input fields for SQL Injection vulnerabilities by monitoring database syntax feedback.

Global Variables

- `payload` (str): The test bypass payload, defined as

```
"' OR '1'='1"
```

Functions

`scan_sqli(url)`

- **Description:** Sends the SQLi payload in the username parameter and parses the response body for common database syntax error signatures.

- **Parameters:**
 - `url` (str): The login endpoint URL (e.g., `http://127.0.0.1:5000/login`).
- **Returns:**
 - `bool`: True if database error signatures are matched, False otherwise.
- **Dependencies:** `requests`
- **Internal Logic:**

12. Define the parameters dictionary:

```
params = {
    "username": payload,
    "password": "test"
}
```

13. Submit the parameters via `requests.get(url, params=params)`.
14. Define a list of signature error tokens: `errors = ["sql", "syntax", "database"]`.
15. Loop through each token. Convert both the token and the response text to lowercase to perform a case-insensitive check: `if error.lower() in response.text.lower()`.
16. Return True if any signature is found.

3.6 Report Generator Module (`report_generator.py`)

Persists findings to disk in a standard format.

Functions

`generate_report(results)`

- **Description:** Writes a list of findings to a local text file.
 - **Parameters:**
 - `results` (list[str]): A list of vulnerability description strings.
 - **Returns:** None.
 - **Internal Logic:**
17. Open the file `report.txt` in write mode ("w") using a `with` context manager.
18. Iterate over the `results` list.
19. Write each item followed by a newline character `\n`.
20. Print "Report Generated" to the console.

3.7 Mock Vulnerable Application (`vulnerable_app/vulnerable.py`)

Exposes a local web app with intentionally vulnerable endpoints to verify the scanner.

Application Routes

- **Route** / (`home()`)

- **Method:** GET
- **Description:** Renders a landing page featuring a search form (action /search, field q) and a login form (action /login, fields username and password).
- **Route** /search (search())
 - **Method:** GET
 - **Description:** Retrieves parameter q from the request arguments. Returns the reflected f-string f"Search Result: {q}" without sanitization.
- **Route** /login (login())
 - **Method:** GET
 - **Description:** Retrieves username and password parameters. If the SQLi payload ' OR '1'='1 is present in the username field, it returns the mock database error "Database Error: SQL Syntax Error". Otherwise, returns "Login Failed".

3.8 Main Scanner Orchestrator (scanner.py)

The execution harness that imports the scanning modules, runs the tests against the local Flask app, collects the results, and writes the final report.

Execution Flow

21. Initialize an empty list: `results = []`.
22. Execute `scan_xss("http://127.0.0.1:5000/search")`. If True, append "[+] XSS Vulnerability Found" to results.
23. Execute `scan_sql("http://127.0.0.1:5000/login")`. If True, append "[+] SQL Injection Found" to results.
24. Call `generate_report(results)` to write findings to report.txt.
25. Print the results list to the console.

4. Unit Test Cases

The following test plan describes how to verify the correctness of the individual modules and their integrations:

Test Case	Description	Pre-requisites	Test Steps	Expected Result
TC-01	Verify Flask App Startup	Python installed, Flask library installed	Run python vulnerable_app/vulnerable.py and access http://127.0.0.1:5000/ via browser or HTTP client	The Flask web application starts successfully and returns the

Test Case	Description	Pre-requisites	Test Steps	Expected Result
				homepage HTML containing the search and login forms.
TC-02	Verify Link Crawling	target application running	Import crawler.py and call <code>get_links("http://127.0.0.1:5000/")</code>	Returns a list containing the links /search and /login.
TC-03	Verify Form Parsing	target application running	Import form_parser.py and call <code>get_forms("http://127.0.0.1:5000/")</code>	Returns a list of BeautifulSoup tag objects matching the two <form> elements in the HTML.
TC-04	Verify XSS Detection	target application running	Call <code>scan_xss("http://127.0.0.1:5000/search")</code>	The function successfully detects the unescaped script reflection and returns True.
TC-05	Verify SQLi Detection	target application running	Call <code>scan_sql("http://127.0.0.1:5000/login")</code>	The function successfully triggers the simulated syntax error and returns True.
TC-06	Verify Report Generation	None	Call <code>generate_report(["[+] XSS Vulnerability Found"])</code>	A file named report.txt is created in the root directory containing the string "[+] XSS Vulnerability Found\n".

Test Case	Description	Pre-requisites	Test Steps	Expected Result
TC-07	Verify Full Scanner Integration	target application running	Run the main orchestrator script python scanner.py	The script executes both scans, prints the findings list containing both vulnerabilities, and generates a valid report.txt file.